

```

1  #include "Matrix3x3.h"
2  #include <sstream>
3  #include <cassert>
4  #include <cmath>
5
6  // Matrix equivalence: Two matrices are equal if all their
   corresponding row vectors are equal.
7  bool Matrix3x3::operator==(const Matrix3x3& aOther) const
   noexcept {
8      return (fRows[0] == aOther.fRows[0]) &&
9              (fRows[1] == aOther.fRows[1]) &&
10             (fRows[2] == aOther.fRows[2]);
11 }
12
13 // Matrix multiplication: Each element (i, j) in the result is
   the dot product of the i-th row of this matrix and the j-th
   column of aOther.
14 Matrix3x3 Matrix3x3::operator*(const Matrix3x3& aOther) const
   noexcept {
15     const Matrix3x3& M = *this;
16     return Matrix3x3(
17         Vector3D( M[0].dot(aOther.column(0)), M[0].dot(aOther.
   column(1)), M[0].dot(aOther.column(2)) ),
18         Vector3D( M[1].dot(aOther.column(0)), M[1].dot(aOther.
   column(1)), M[1].dot(aOther.column(2)) ),
19         Vector3D( M[2].dot(aOther.column(0)), M[2].dot(aOther.
   column(1)), M[2].dot(aOther.column(2)) )
20     );
21 }
22
23 // Transpose: Create a new matrix where each row is a column of
   the original matrix.
24 Matrix3x3 Matrix3x3::transpose() const noexcept {
25     return Matrix3x3(
26         column(0),
27         column(1),
28         column(2)
29     );
30 }
31
32 // Determinant: Compute the determinant using the standard 3x3
   formula.
33 float Matrix3x3::det() const noexcept {
34     const Matrix3x3& M = *this;
35     float M11 = M[0][0], M12 = M[0][1], M13 = M[0][2];
36     float M21 = M[1][0], M22 = M[1][1], M23 = M[1][2];
37     float M31 = M[2][0], M32 = M[2][1], M33 = M[2][2];
38     return M11 * (M22 * M33 - M23 * M32)

```

```

39         - M12 * (M21 * M33 - M23 * M31)
40         + M13 * (M21 * M32 - M22 * M31);
41     }
42
43     // Invertibility check: A matrix is invertible if the absolute
value of its determinant is greater than eps.
44     bool Matrix3x3::hasInverse() const noexcept {
45         return std::fabs(det()) > eps;
46     }
47
48     // Inverse:
49     // First compute the cofactors
50     // Then form the transpose of the cofactor matrix
51     // After that multiply by 1/det.
52     Matrix3x3 Matrix3x3::inverse() const noexcept {
53         const Matrix3x3& M = *this;
54         float determinant = det();
55         assert(std::fabs(determinant) > eps); // assert if the
matrix is invertible.
56
57         // Compute cofactors (with alternating signs)
58         float c00 = (M[1][1] * M[2][2] - M[1][2] * M[2][1]);
59         float c01 = -(M[1][0] * M[2][2] - M[1][2] * M[2][0]);
60         float c02 = (M[1][0] * M[2][1] - M[1][1] * M[2][0]);
61
62         float c10 = -(M[0][1] * M[2][2] - M[0][2] * M[2][1]);
63         float c11 = (M[0][0] * M[2][2] - M[0][2] * M[2][0]);
64         float c12 = -(M[0][0] * M[2][1] - M[0][1] * M[2][0]);
65
66         float c20 = (M[0][1] * M[1][2] - M[0][2] * M[1][1]);
67         float c21 = -(M[0][0] * M[1][2] - M[0][2] * M[1][0]);
68         float c22 = (M[0][0] * M[1][1] - M[0][1] * M[1][0]);
69
70         // Adjugate is the transpose of the cofactor matrix.
71         Matrix3x3 adjugate(
72             Vector3D(c00, c10, c20),
73             Vector3D(c01, c11, c21),
74             Vector3D(c02, c12, c22)
75         );
76
77         return adjugate * (1.0f / determinant);
78     }
79
80     // Output operator: Format the matrix as [[row1],[row2],[row3
]] using Vector3D's toString() method
81     std::ostream& operator<<(std::ostream& aOStream, const
82         Matrix3x3& aMatrix) {
83         aOStream << "["

```

```
83         << aMatrix[0].toString() << ","  
84         << aMatrix[1].toString() << ","  
85         << aMatrix[2].toString() << "];  
86     return aOStream;  
87 }  
88
```